MVCC

什么是MVCC? 为什么要有MVCC?

一张数据库表里面存了很多数据. 现在有很多并发事务来访问或者修改数据. 在并发事务情况下会出现脏读, 不可重复读, 幻读问题.

你要解决脏读, 就得上读已提交的隔离级别. 也就是说, 你得想办法保证一个事务只能读到其他事务已提交的数据, 其他事务没提交的数据, 不应该让它读到.

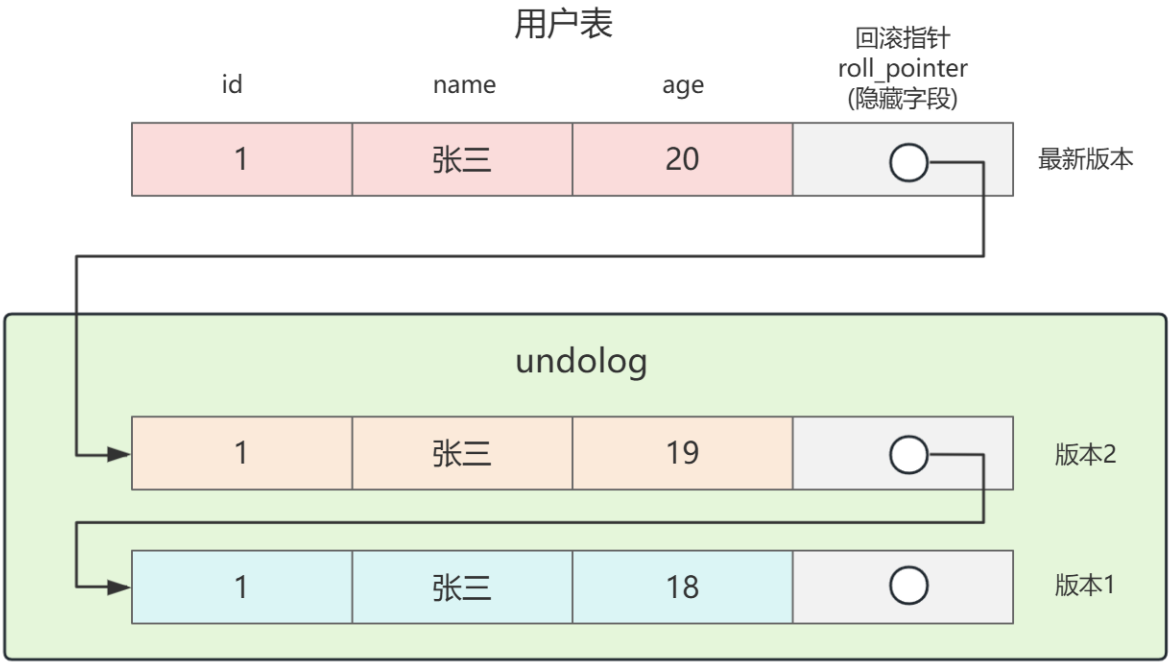
你要解决不可重复读, 就得上可重复读隔离级别. 也就是说一个事务, 第一次读取之后, 后面每次读到的数据都应该和第一次一样, 即便后面有其他事务新提交了数据, 也不应该读到.

那怎么实现呢? 要怎么设计呢?

这就需要维护一份数据的多个版本, 每个事务修改一次, 就得生成一个版本, 让不同的事务去读不同的版本. 在读已提交的隔离级别下, 让事务去读已经提交的数据版本, 这样就能避免脏读.

在不可重复读的隔离级别下, 让事务每次都读同一个版本的数据, 这样每次读到的都一样了, 就能避免可重复读的问题.

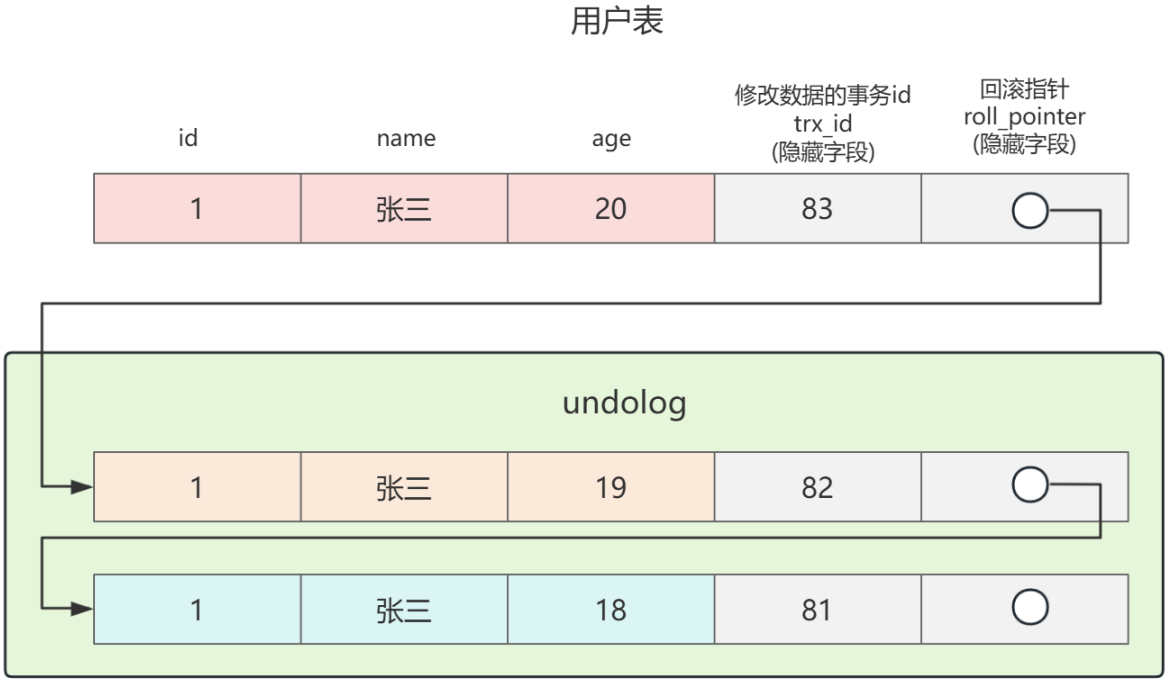
所以可以把修改的数据版本记录到undolog日志中, 然后给表增加一个隐藏字段, 回滚指针, 让回滚指针指向undolog日志, 使用回滚指针将历史版本串联为一个链表. 这样的话, 想读哪个历史版本, 沿着链表一直找就可以了.



那么问题来了, 一个事务来查数据, 怎么知道我要查哪个版本的数据呢?

给每个事务分配一个事务id, 事务id自增分配. 通过对比事务id的大小, 我就能知道哪个事务创建早, 哪个事务创建的晚. 那创建比较晚的事务修改的数据, 别让创建早的事务看到就行了.

所以我们需要给表再增加一个隐藏字段, 修改数据的事务id, 谁修改了它, 就把对应的事务id记录下来.

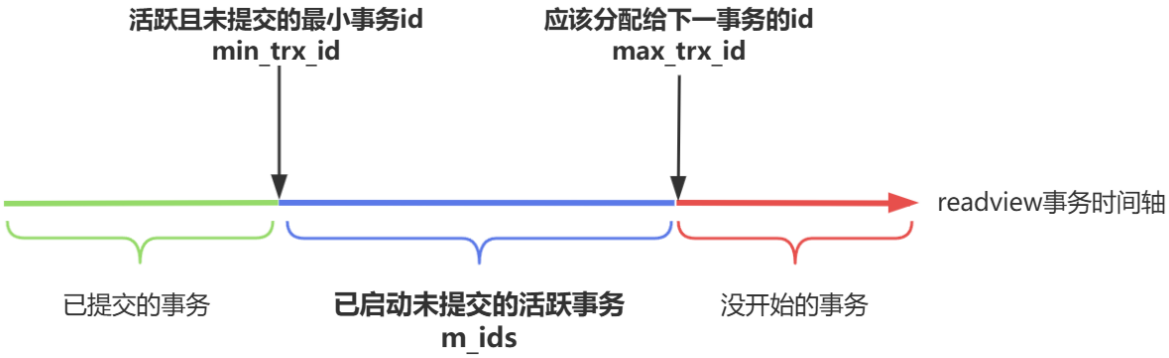


还需要一个东西, 叫readview. readview中有4个重要字段:

- creator_trx_id: 创建当前readview的事务id
- m_ids: 创建readview时, 当前数据库中活跃且未提交的所有事务id
- min_trx_id: 创建readview时, 数据库活跃且未提交的最小事务id
- max_trx_id: 创建readview时, 应该分配给下一个新建事务的id

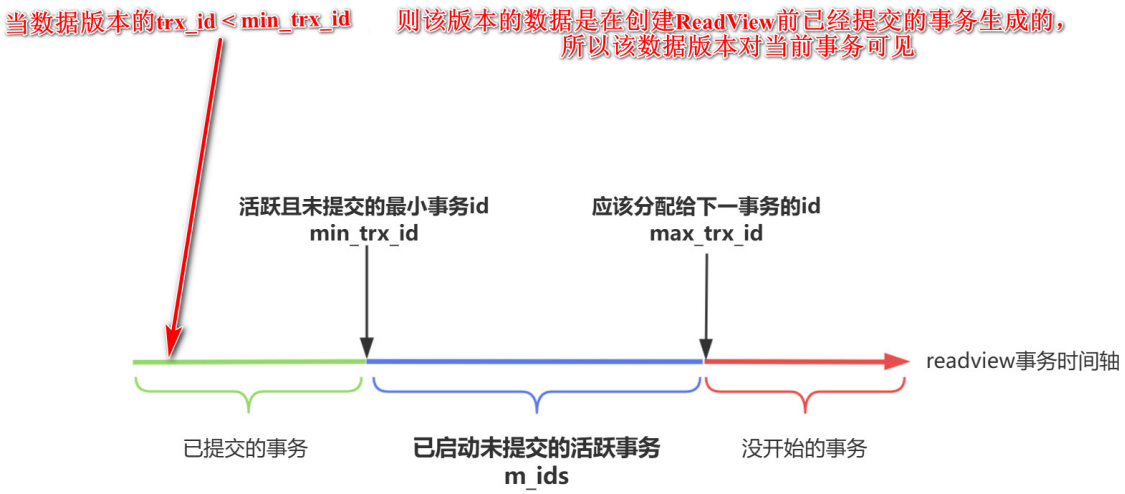
那这些字段有啥用啊?

看图, readview的本质就是描绘了一个创建当前事务时的事务id数轴. 通过对比数据版本的trx_id在数轴的哪个位置, 就能知道这个数据版本是否对当前事务可见.

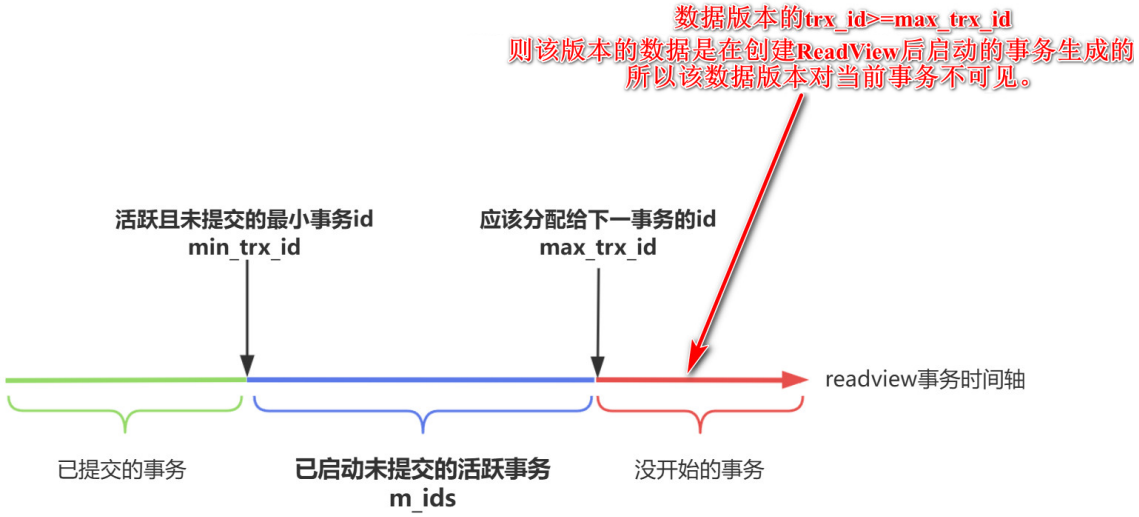


主要有这么几种情况.

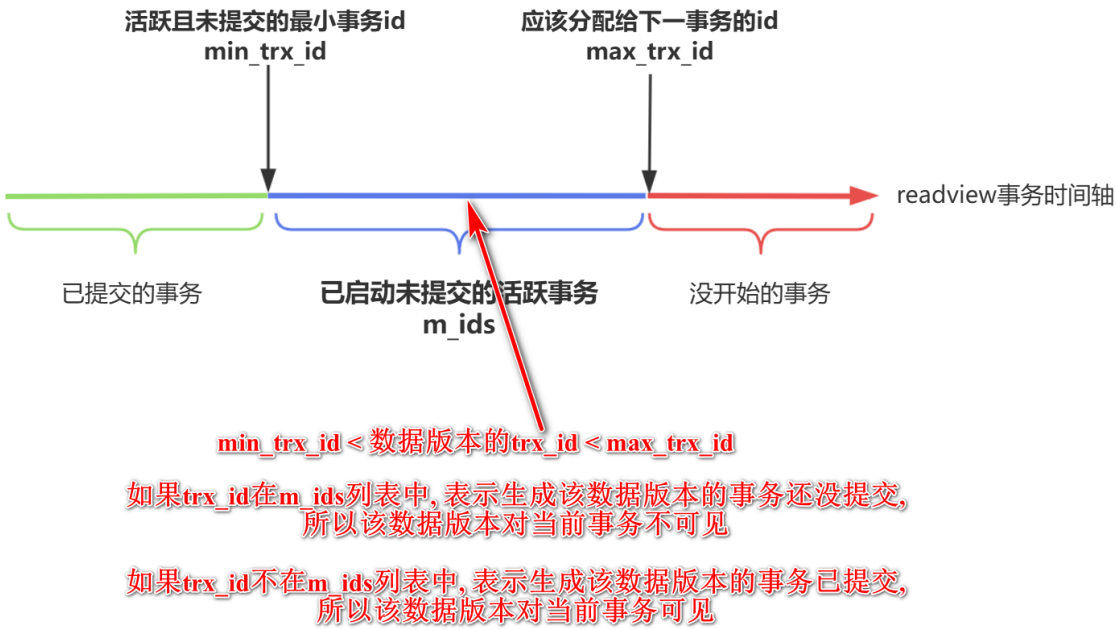
- 如果一条数据的trx_id值小于ReadView中的min_trx_id值, 比未提交的最小事务id还小, 这就说明这个数据版本在创建ReadView前就生成了, 这数据早都有了, 所以这个数据版本对当前事务是可见的.



- 如果记录的trx_id值大于等于ReadView中的max_trx_id值，说明这个数据版本是在创建ReadView后才启动的某个事务生成的, 这是新启动的事务创建的数据, 所以这个数据版本对当前事务不可见.



- 如果记录的trx_id值在ReadView的min_trx_id和max_trx_id之间, 就得分情况了
  - 如果这个数据版本trx_id在m_ids列表中，说明生成这个数据版本的事务没提交, 都没提交, 能让当前事务看到吗, 肯定不能啊. 所以就不可见
  - 相反, 如果这个数据版本trx_id不在m_ids列表中, 说明生成这个数据版本的事务已经提交了, 已经提交了, 肯定就能看了, 所以这个数据版本对当前事务可见的.



所以一句话总结mvcc, 每次修改数据就记录修改的事务id到隐藏字段, 然后生成一个版本记录到undolog版本链中, 然后回滚指针指向版本链, 通过对比readview和数据版本的事务id, 就能知道某个版本的数据对当前事务是否可见.

所以undo log记录数据版本, readview来判断数据版本对当前事务的可见性, 这就是MVCC.

MVCC搞定了, 那怎么实现各个隔离级别呢?

首先读未提交怎么实现?

- 不用实现, 你根本不用管, 多个并发事务同时执行天然就是读未提交.
- 当然这里说的是读的时候你不用管, 写的时候还是要加锁的, 不能同时写一个数据呀.

那串行化怎么实现的?

- 直接加锁. 只有能拿到锁的事务才能执行, 这样事务就串行化执行了.

那么问题来了, 可重复读怎么实现的?

- 可重复读, 每次都会读到和第一次相同的数据, 即便后面有新事务修改了数据, 还是会读到和第一次相同的数据, 所以解决了不可重复读的问题.
- 那么问题来了, 怎么让他每次都读到相同的数据呢? 可重复读会在第一次select的时候生成一个readview, 刚说了, readview是用来判断数据版本对当前事务的可见性的, 可重复读每次都会复用第一次生成的readview.
- 也就是说, 当生成这个readview的时候, 哪些版本的数据对当前事务可见就已经固定下来了, 后续每次通过同一个readview判断数据可见性, 所以每次都读的是同一个数据版本.
- 所以可重复读就是通过MVCC+复用第一次生成的readview来实现的. 除此之外, 可重复读为了避免幻读问题, 还会加锁. 这个后面详细讲.

读已提交怎么实现的?

- 读已提交就是说, 第一次读一些数据, 后面只要有事务修改了数据, 然后提交了, 那再读就能读到最新的已经提交的数据.
- 读已提交的本质就是每次select就生成一个新的readview, 每次读都生成新的readview, 就能每次读到新提交的数据版本.

在可重复读的隔离级别怎么解决幻读的呢?

MySQL默认是可重复读的隔离级别, 可重复读能解决脏读和不可重复读的问题, 那幻读问题怎么解决?

- 串行化虽然能解决幻读, 但是性能太差, 不推荐用.
- 所以只能考虑在可重复读的隔离级别解决幻读问题.

那在可重复读的隔离级别怎么解决幻读的呢?

这里分为两种情况, 第一种, 快照读.

- 在可重复读的隔离级别下, 当执行普通的select语句, 会通过MVCC去读, 每次都会读同一个数据版本, 这就是快照读. 普通select通过快照读去避免加锁, 每次都读快照, 别的事务干了啥, 我读不到, 就不会幻读.
- 但是你不能永远读不到吧, 比如你先开启事务, 别人后开启事务, 但是别人先插了id为1的数据, 你读不到, 然后你以为id为1的数据不存在, 你就插入id为1的数据, 能插吗? 肯定不能插. 所以插入这种操作必须读到最新数据才能判断能不能让你插入. 这就是当前读.

当前读

- 执行select for update, 或者select in share mode, 或者增删改操作, 必须要读到最新版本的数据, 这就是当前读.
- 当前读 读的都是最新的数据, 如果其它事务插入了, 那不就还会产生幻读吗? 怎么办?
- 执行当前读, MySQL会使用临键锁去锁住数据, 和数据之间的间隙. 我加锁了, 你就不能插入了, 只要不让你不插入, 我就不会发生幻读.
- 举个例子, `select 字段 from 某张表 where id >= 100 for update`, 查询id大于等于100的数据, 这里有for update, 所以是当前读. MySQL会锁住id=100这条数据, 其他事务就不能修改了, 然后再锁住100到正无穷这个区间, 不让你插入id大于100的数据. 只要不让你插入, 我就不会幻读.

- ```
  mysql> begin;
  Query OK, 0 rows affected (0.00 sec)

  mysql> select * from test1 where id >= 100 for update;
  +-----+----------+------+
  | id  | username | age  |
  +-----+----------+------+
  | 100 | ddd      |   33 |
  +-----+----------+------+
  1 row in set (0.00 sec)
  ```
  事务A先开启事务

  以当前读查询id>=100的数据, 加临键锁

  ```
  mysql> begin;
  Query OK, 0 rows affected (0.00 sec)

  mysql> insert into test1 values (101, 'eee', 44);
  ```
  事务B后开启事务

  插入id=101的数据直接阻塞

所以, 在可重复读的隔离级别, 是通过MVCC快照读+当前读加锁的方式方式幻读的.

那么问题来了, 这样就真的能完全避免幻读吗? 真的能吗?

很可惜, 并不能. MVCC+锁只能避免大多数的幻读问题, 极端情况下还是会出现幻读.

什么情况下还会产生幻读

什么情况会出现幻读呢?

- 先快照读, 再当前读的情况下就会出现幻读.

举个例子

- 事务A执行普通select查id=1的数据, 表里没有, 查不出来
- 然后事务B插入id=1的数据, 并提交事务.
- 事务A再用普通select查id=1的数据, 还是查不出来, 因为是快照读. 但是事务A使用select for update当前读去查, 就能查到id=1的数据. 先快照读, 再当前读就出现了幻读.
  ```

- 事务A先开启事务
普通select查询id=1的数据, 表中没有, 查不到

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from test1 where id = 1;
Empty set (0.00 sec)
```

事务B后开启事务
插入id=1的数据

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into test1 values (1, 'fff', 12);
Query OK, 1 row affected (0.00 sec)

mysql> commit
    -> ;
Query OK, 0 rows affected (0.00 sec)
```

事务A再次普通select查询id=1的数据
因为是快照读, 所以还是查不到

```
mysql> select * from test1 where id = 1;
Empty set (0.00 sec)

mysql> select * from test1 where id = 1 for update;
+----+----------+------+
| id | username | age  |
+----+----------+------+
|  1 | fff      |   12 |
+----+----------+------+
1 row in set (0.00 sec)
```

事务A执行select for update查询id=1的数据
因为是当前读, 所以就查到了

此时就发生了幻读

再举个更诡异的例子. 我们系统中用户名是不允许重复的, 所以给用户名加唯一索引. 然后一般先从数据库中查这个用户名是否存在, 不存在才能插入. 这事务并发执行就是这样的.

- 事务A使用普通select查询是否存在用户名为aaa的用户, 表里没有, 查不到
- 然后事务B插入了用户名为aaa的用户, 并提交事务.
- 事务A插入用户名=aaa, 插入不了, 提示用户名=aaa已存在.
- 事务A再使用普通select查询用户名=aaa的数据, 因为是快照读, 所以还是查不到.
- 发现问题了吗, 我查了明明没有用户名为aaa的数据, 但是一插入就提示已存在. 这就是幻读
- 但是更骚的来了, 事务A查不到aaa, 但是直接update修改用户名为aaa的数据. 就是这么诡异, 你查不到, 无法插入, 但是可以改, 就是这么诡异.
- 改完后再去查就能查到了.

这又是个先快照读, 再当前读产生的幻读.

事务A先开启事务
查询username=AAA的数据, 发现没有

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from test1 where username='AAA';
Empty set (0.00 sec)
```

事务B后开启事务
插入username=AAA的数据

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into test1 values (200, 'AAA', 55);
Query OK, 1 row affected (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.00 sec)
```

事务A插入username=AAA的数据
提示AAA已存在 (因为有唯一索引)
但是查询却查不到 (快照读)

```
mysql> insert into test1 values (300, 'AAA', 33);
ERROR 1062 (23000): Duplicate entry 'AAA' for key 'username'
mysql> select * from test1 where username='AAA';
Empty set (0.00 sec)
```

```
mysql> update test1 set age=88 where username='AAA';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from test1 where username='AAA';
+-----+----------+------+
| id  | username | age  |
+-----+----------+------+
| 200 | AAA      |   88 |
+-----+----------+------+
1 row in set (0.00 sec)
```

事务A查不到username=AAA的数据, 但却可以修改!!!
修改之后就能查到username=AAA的数据了

怎么解决幻读

那咋办呢? 怎么解决这种幻读问题?

其实就一点, 在一个事务在操作某张表数据的时候, 另外一个事务不允许新增或者删除这张表中的数据了

- 在可重复读的事务级别下, 要么给表加表锁, 要么给数据加行锁.

怎么加呢? 就是使用select for update.

```
# 加表锁:

# 事务1:
select * from 表 for update; # 执行成功, 加表锁

# 事务2:
select * from 表 for update; # 阻塞
insert into 表;  # 阻塞
```

```
# 加行锁:

# 事务1:
select * from 表 where 索引列=1 for update; # 执行成功, 加行锁

# 事务2:
select * from 表 where 索引列=1 for update; # 阻塞
insert into 表 values; # 阻塞
```